

FAST National University of Computer and Emerging Sciences

Chiniot-Faisalabad Campus (CFD)

Department of Computer Science



CHESS GAME



C++ Edition — Object Oriented Programming

Submitted By	Shahryar (25F-0004)
	Ali Mardan (25F-0120)
	Ali Mumtaz (25F-0012)
Section	BS AI-2B
Course	Object Oriented Programming (OOP)
Instructor	Sir Hassan Sami
University	FAST NUCES — Chiniot-Faisalabad
Submission	May 2026

Table of Contents

1. Introduction
2. Project Objectives
3. OOP Concepts Applied
4. Project Structure — Files & Classes
5. Detailed Class Descriptions
6. Game Features
7. Program Output — Screenshots
8. Challenges & Solutions
9. Conclusion

1. Introduction

Chess is one of the oldest and most intellectually demanding strategy games in the world, played by millions across all cultures and age groups. This project implements a fully functional two-player Chess Game in C++ using the principles of Object Oriented Programming (OOP). The game runs in the Windows console, leveraging the Windows Console API to deliver a visually rich and interactive experience — with color-coded pieces, a clearly labeled board, turn indicators, check warnings, and a game-over screen.

This project was developed as a semester assignment for the Object Oriented Programming course at FAST NUCES Chiniot-Faisalabad Campus. It applies real-world OOP design: an abstract base class for pieces, concrete derived classes for each of the six chess piece types, a board manager class, and a game controller class — all working together through inheritance and polymorphism.

Players interact using standard chess notation — typing the source square (e.g., e2) and destination square (e.g., e4). The program validates every move, enforces turn order, warns when a King is in check, and declares the winner when a King is captured.

2. Project Objectives

- ▶ Implement a complete, playable two-player Chess game in C++ using OOP design.
- ▶ Demonstrate inheritance through a Piece abstract base class and six derived piece classes.
- ▶ Apply runtime polymorphism via virtual functions for move validation and piece identification.
- ▶ Use encapsulation to protect all internal state through private data members and public interfaces.
- ▶ Render a colored, readable console board using the Windows Console API (no external libraries).
- ▶ Provide a styled main menu for starting a new game or exiting.
- ▶ Validate all player input and reject illegal moves with clear, color-coded error messages.
- ▶ Detect and warn about check conditions and declare a winner when a King is captured.
- ▶ Use dynamic memory allocation (new/delete) with proper destruction to prevent memory leaks.

3. OOP Concepts Applied

3.1 Inheritance

The Piece class is the abstract base class for all chess pieces. Six concrete classes — King, Queen, Rook, Bishop, Knight, and Pawn — inherit from it. Each derived class receives the common attributes (color, x, y) and the shared interface (getcolor, getx, gety, setposition) from Piece, while providing its own implementation of movement rules. This eliminates code duplication and establishes a clean class hierarchy.

3.2 Polymorphism (Runtime)

isvalidmove(toX, toY, board) and getsymbol() are declared as pure virtual functions in Piece. The ChessBoard class stores Piece* pointers in its 8x8 grid. When movePiece() is called, it invokes piece->isvalidmove() without knowing the actual piece type — the correct overridden version is called at runtime based on the actual object type. This is classic runtime polymorphism through virtual dispatch.

3.3 Abstraction

Because Piece contains pure virtual functions it is an abstract class and cannot be instantiated directly. It defines what every piece must be able to do (validate a move, return a symbol) without specifying how. The implementation details are hidden inside each specific piece class, abstracting away complexity from the ChessBoard and Game classes.

3.4 Encapsulation

All data members across all classes are private: grid[][] in ChessBoard, color/x/y in Piece, currentTurn in Game. They are accessed and modified only through well-defined public functions: getcolor(), setposition(), getPiece(), movePiece(), isKingAlive(). This protects the internal state of every object from accidental external modification.

3.5 Dynamic Memory Allocation & Destructors

All 32 Piece objects are created on the heap using new in the ChessBoard constructor. When a piece is captured, the old piece at the destination is deleted before the new piece moves there. The ChessBoard destructor iterates through all 64 cells and calls delete on every non-null pointer, ensuring zero memory leaks when the game exits.

4. Project Structure — Files & Classes

File / Header	Responsibility
main.cpp	Entry point. Creates a Game object and calls game.start().
piece.h / piece.cpp	Abstract base class. Declares pure virtual isvalidmove() and getsymbol().
chessboard.h / .cpp	Owns the 8×8 grid. Handles initialization, display, move execution, and king detection.
game.h / game.cpp	Top-level controller: main menu, game loop, turn switching, input parsing, check warning.
king.h / king.cpp	King — one square in any of the 8 directions.
queen.h / queen.cpp	Queen — any number of squares straight or diagonal.
rook.h / rook.cpp	Rook — any number of squares horizontally or vertically.
bishop.h / bishop.cpp	Bishop — any number of squares diagonally.
knight.h / knight.cpp	Knight — L-shaped move; can jump over other pieces.
pawn.h / pawn.cpp	Pawn — one square forward; captures diagonally.

5. Detailed Class Descriptions

5.1 Piece (Abstract Base Class)

Piece is the root of the chess piece hierarchy. It stores:

- ▶ char color — W for White, B for Black.
- ▶ int x, int y — current row and column on the board.

It declares two pure virtual functions that every subclass must override:

- ▶ virtual bool isvalidmove(int toX, int toY, Piece* board[8][8]) = 0
- ▶ virtual char getsymbol() const = 0

Concrete helper methods: getcolor(), getx(), gety(), setposition(). Virtual destructor ensures correct cleanup through base pointers.

5.2 ChessBoard

ChessBoard owns and manages a Piece* grid[8][8]. Key responsibilities:

- ▶ Constructor — allocates all 32 pieces with new and places them at their standard starting positions.
- ▶ display() — renders the board to the console using Windows Console API color attributes: alternating dark/light squares, RED text for White pieces, BLUE text for Black pieces, GREEN labels for coordinates and legend.
- ▶ movePiece(fromX, fromY, toX, toY, currentTurn) — checks boundaries, verifies piece color matches the current turn, calls isvalidmove() polymorphically, deletes the captured piece if any, and updates the grid.
- ▶ isKingAlive(color) — scans the entire grid to find the King of the given color.
- ▶ Destructor — iterates all 64 cells, calls delete on every non-null pointer.

5.3 Game

Game is the top-level controller that orchestrates the entire session:

- ▶ showMenu() — displays the styled main menu with yellow-highlighted options.
- ▶ start() — clears the screen with system("cls"), redraws the board, reads From/To input in chess notation, parses it to array indices, calls board.movePiece(), detects check, and switches turns.
- ▶ switchTurn() — toggles currentTurn between W and B after every valid move.

- ▶ `isGameOver()` — calls `board.isKingAlive()` for both colors; prints a winner message and returns true if a King is missing.

5.4 Piece Subclasses

Each of the six subclasses implements `isvalidmove()` according to standard chess rules:

- ▶ King — checks that the destination is exactly one step away in any direction and not occupied by a friendly piece.
- ▶ Queen — combines Rook and Bishop logic: clear path along ranks, files, or diagonals.
- ▶ Rook — requires the move to be on the same row or column with no blocking pieces in between.
- ▶ Bishop — requires a diagonal move with no blocking pieces along the diagonal.
- ▶ Knight — checks the eight possible L-shaped destinations; can jump over other pieces.
- ▶ Pawn — moves one square forward; captures one square diagonally; no backward movement.

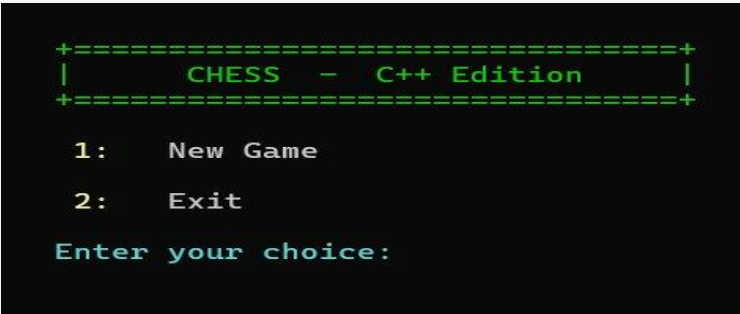
6. Game Features

- ▶ Colored console board — alternating dark/light squares via Windows Console API background attributes.
- ▶ Color-coded pieces — RED for White side, BLUE for Black side, GREEN for board labels and borders.
- ▶ Chess notation input — moves entered as column letter + row number (e.g., f2 then f3).
- ▶ Input validation — wrong format, out-of-range squares, and empty squares are all rejected gracefully.
- ▶ Move validation — each piece enforces its own legal movement rules via polymorphic `isvalidmove()`.
- ▶ Automatic turn switching — alternates between WHITE and BLACK after every successful move.
- ▶ Check detection — warns the active player in yellow when their King is under threat.
- ▶ King capture detection — the game ends and a winner is declared when a King is taken.
- ▶ Styled main menu — color-coded options with a framed banner title.
- ▶ screen clearing (`system("cls")`) — the board is redrawn cleanly before every turn.
- ▶ Quit command — players can type quit at any time to exit the game cleanly.
- ▶ Game over screen — a framed GAME OVER banner is shown with a press-Enter prompt.

7. Program Output — Screenshots

7.1 Main Menu

On launch the program displays a styled main menu. Menu options are highlighted in yellow. The player presses 1 to start a new game or 2 to exit.



```
+=====+
|      CHESS  -  C++ Edition      |
+=====+

  1:   New Game

  2:   Exit

Enter your choice:
```

Figure 1: Main Menu Screen

7.2 Chess Board — Initial State

After selecting New Game the board is displayed in its standard starting position. BLUE letters represent Black pieces (rows 7–8) and RED letters represent White pieces (rows 1–2). The board uses alternating dark and light squares with coordinate labels on all four sides.

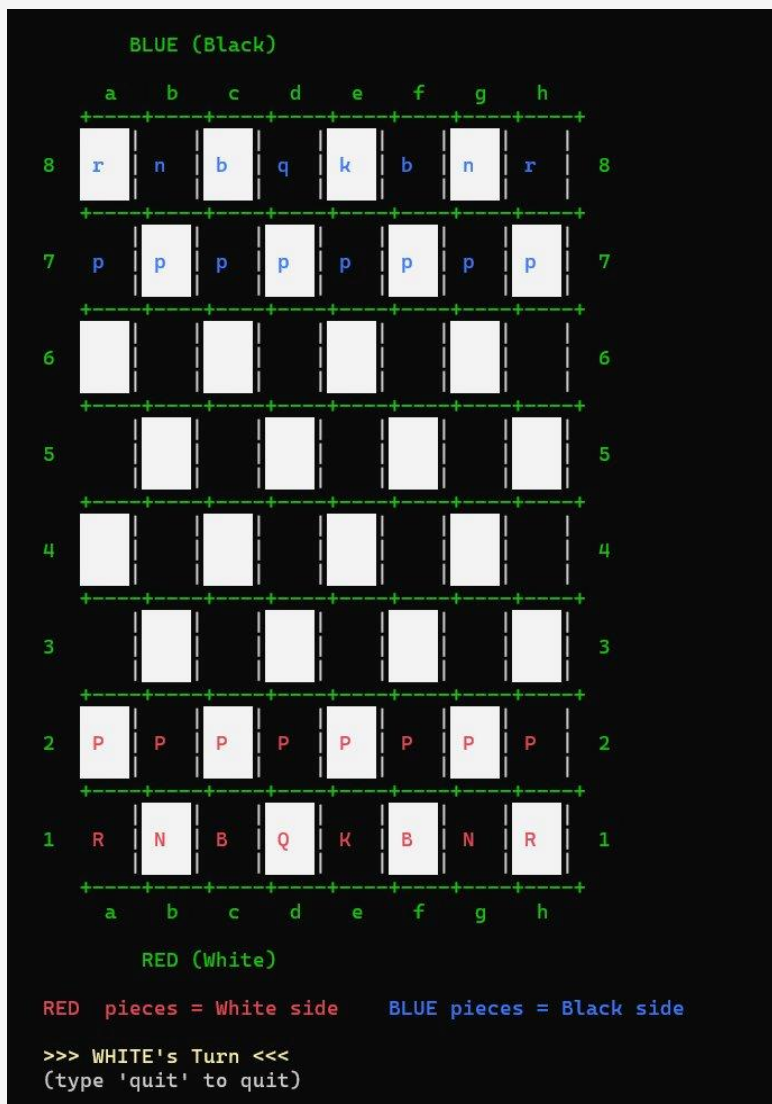


Figure 2: Chess Board at Game Start

7.3 Gameplay in Progress

As the game progresses pieces move across the board. Both sides have made several moves — Black pieces (BLUE) and White pieces (RED) are spread across the board. The player enters moves using chess notation (From: f2 / To: f3).

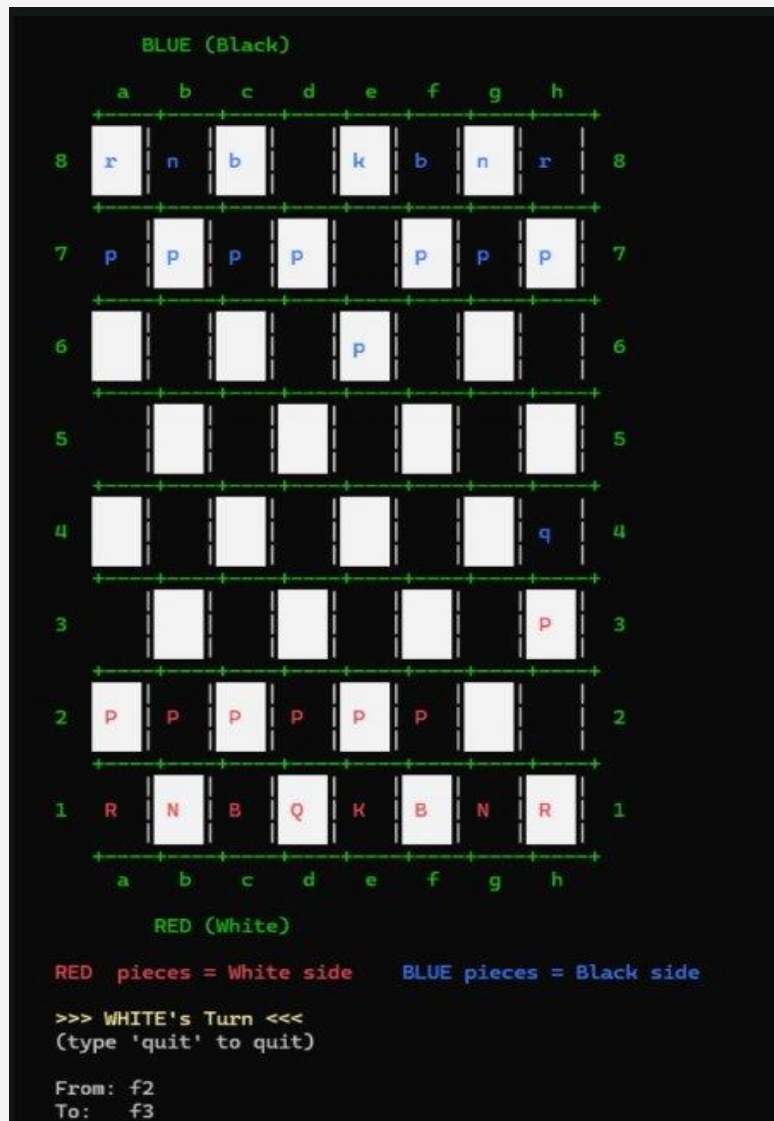


Figure 3: Gameplay — Several Moves In

7.4 Check Warning

When the active player's King is under threat the game displays a yellow warning: "*** WARNING: Your king is in CHECK! ***". The player must respond to the check on their next move.

```

BLUE (Black)
  a  b  c  d  e  f  g  h
8  r  n  b      k      n  r  8
7  p  p  p  p      p  p  p  7
6      p      p      p      6
5      b      p      p      5
4      p      p      p      4
3  p      p      p      p  3
2      P  P  P  P  q  R  2
1  R  N  B  Q  K  B  N  1
  a  b  c  d  e  f  g  h

RED (White)

RED pieces = White side    BLUE pieces = Black side

*** WARNING: Your king is in CHECK! ***
>>> WHITE's Turn <<<
(type 'quit' to quit)

From: |

```

Figure 4: Check Warning Displayed in Yellow

7.5 Invalid Move Detection

When a player attempts an illegal move — such as moving a piece to a square it cannot legally reach — the game displays "Invalid move! Try again." in red. The board is not updated and the same player must enter a new move.

```

8  r  n  b  q  k  b  n  r  8
+---+---+---+---+---+---+---+
7  p  p  p  p  p  p  p  p  7
+---+---+---+---+---+---+---+
6  p  p  p  p  p  p  p  p  6
+---+---+---+---+---+---+---+
5  p  p  p  p  p  p  p  p  5
+---+---+---+---+---+---+---+
4  p  p  p  p  p  p  p  p  4
+---+---+---+---+---+---+---+
3  P  P  P  P  P  P  P  P  3
+---+---+---+---+---+---+---+
2  P  P  P  P  P  P  P  P  2
+---+---+---+---+---+---+---+
1  R  N  B  Q  K  B  N  R  1
+---+---+---+---+---+---+---+
  a  b  c  d  e  f  g  h

RED (White)

RED pieces = White side    BLUE pieces = Black side

>>> BLACK's Turn <<<
(type 'quit' to quit)

From: e1
To:   e2

Invalid move! Try again.

```

Figure 5: Invalid Move — Red Error Message

7.6 Black Wins

After Black captures the White King the game ends immediately. The board is displayed one final time in its end state, and the program prints "**** BLACK WINS! ****" below the board. White's King is gone from the board.

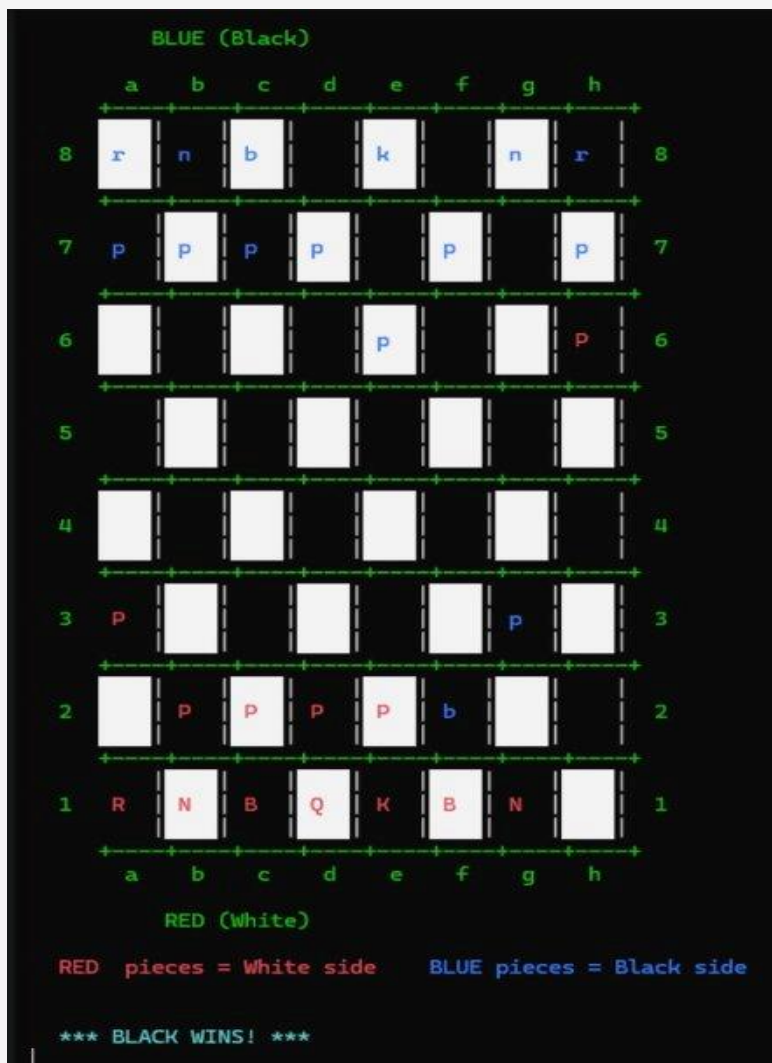


Figure 6: Black Wins — White King Captured

7.7 Game Over Screen

After the winner is announced, a framed GAME OVER banner is shown. The program waits for the player to press Enter before closing, giving them time to see the final result.



Figure 7: Game Over Screen

8. Challenges & Solutions

8.1 SFML Library Configuration

The project was initially planned to use SFML for a graphical window. Configuring SFML in Visual Studio required setting three separate paths — Additional Include Directories, Additional Library Directories, and copying DLLs to the output folder. The SFML folder resided inside an OneDrive directory with a space in the username (C:\Users\Shahryar Hameed\OneDrive\Desktop\...) which caused all paths to fail silently. Additionally, library names were pasted into the linker without semicolons, causing LNK1181 errors.

Solution: The exact path was found by clicking the address bar in File Explorer and copying it verbatim. Library names were corrected with semicolons. After resolving these issues, the approach was switched to the Windows Console API for reliability and zero external dependencies.

8.2 Unicode Chess Symbols

An attempt was made to use Unicode chess piece symbols (♔ ♚ ♜ ♞ ♝ ♟) in the console display. This produced numerous C4566 compiler warnings because the default Windows code page 1252 does not support those Unicode code points.

Solution: Replaced all Unicode symbols with plain ASCII letters (K, Q, R, B, N, P) rendered with colored console attributes. This works on all Windows systems without any encoding configuration and is arguably more readable in a monospace console font.

8.3 Move Validation Logic

Implementing legally correct move validation for all six piece types required careful planning. The Pawn was the most complex case — it moves forward for normal moves but diagonally for captures, and cannot move backward. The Knight required checking exactly eight possible L-shaped destinations. All sliding pieces (Queen, Rook, Bishop) needed to verify that no blocking pieces exist along their path.

Solution: Each piece's `isvalidmove()` was implemented and tested independently with a series of manual test moves before integrating into the full game.

8.4 Coordinate System Mapping

Standard chess notation uses column letters (a–h) and row numbers (1–8) where row 8 is at the top of the board. The internal 2D array uses row index 0 at the top. Mapping these two systems required the formula: $\text{arrayRow} = 8 - \text{rowNumber}$, $\text{arrayCol} = \text{letter} - 'a'$. Getting this wrong initially caused moves to land on the wrong squares.

Solution: The conversion formula was isolated into the input-parsing section of `Game::start()` and verified with multiple test cases before connecting it to `movePiece()`.

9. Conclusion

This project successfully delivers a complete, playable two-player Chess game in C++ that demonstrates all four pillars of Object Oriented Programming. The abstract Piece hierarchy, polymorphic move validation, encapsulated board management, and dynamic memory handling together form a clean, maintainable, and extensible codebase.

The colored console interface — rendered with the Windows Console API — provides a genuinely readable and visually organized playing experience: alternating board squares, color-coded pieces for each side, coordinate labels, turn indicators, check warnings, and a game-over screen. All of this was achieved without any external graphics libraries, making the project easy to build and run on any Windows machine with Visual Studio.

The challenges encountered during development — particularly SFML configuration, Unicode encoding, and move validation logic — were valuable learning experiences that deepened understanding of the C++ toolchain, the Visual Studio build system, and the importance of careful coordinate system design.

Looking ahead, the project could be extended with: check and checkmate detection (currently the game ends only on King capture), special moves such as castling, en passant, and pawn promotion, an AI opponent using minimax search, and a full graphical interface using SFML or Qt. The current OOP structure makes all these extensions straightforward to add without restructuring the core codebase.

This project was developed by Shahryar (25F-0004), Ali Mardan (25F-0120), and Ali Mumtaz (25F-0012) as part of the Object Oriented Programming course under the guidance of Sir Hassan Sami at FAST NUCES Chiniot-Faisalabad Campus, May 2026.